

파이썬 PC 애플리케이션 개발

2025년 11월 07일 조수연

Pyinstaller

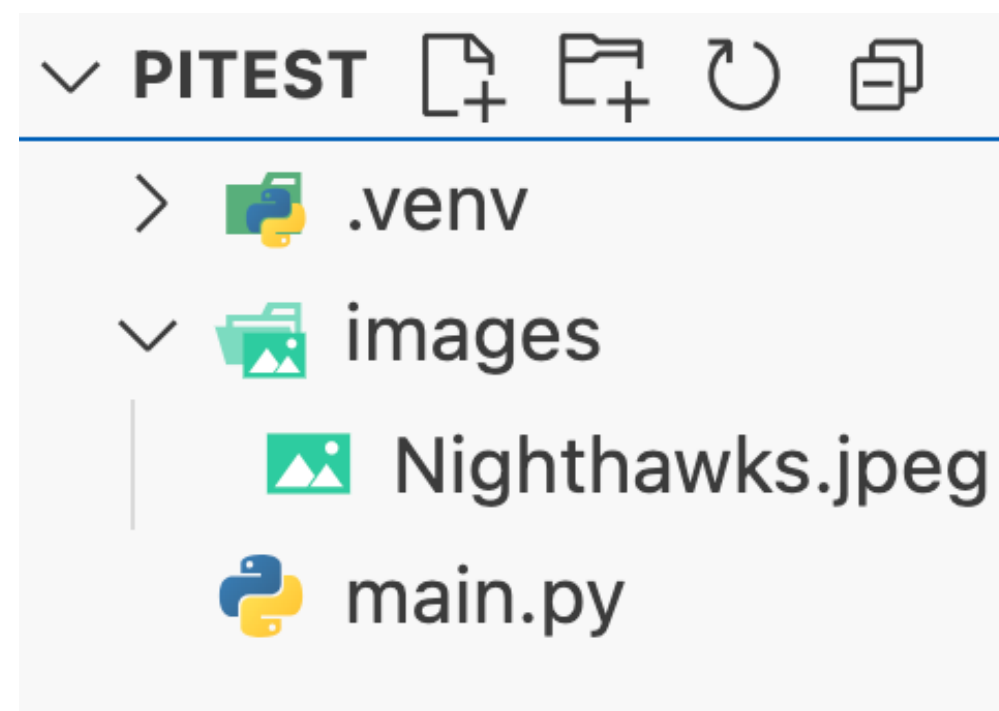
Pyinstaller

<https://pyinstaller.org/en/stable/>

- PC 환경에서 실행되는 파이썬 애플리케이션을 빌드해준다.
 - Windows, macOS, Linux
- 가상환경과 함께 사용하면 파이썬이 설치되지 않은 PC에서도 실행된다.

새 폴더 만들고 가상환경 설정

- PITest 폴더 만들고 VSCode 에서 폴더 열기
`pip install pyinstaller matplotlib`
- 프로젝트에 images 폴더 추가 후 png나 jpg 추가
- 프로젝트 루트에 main.py 파일 추가



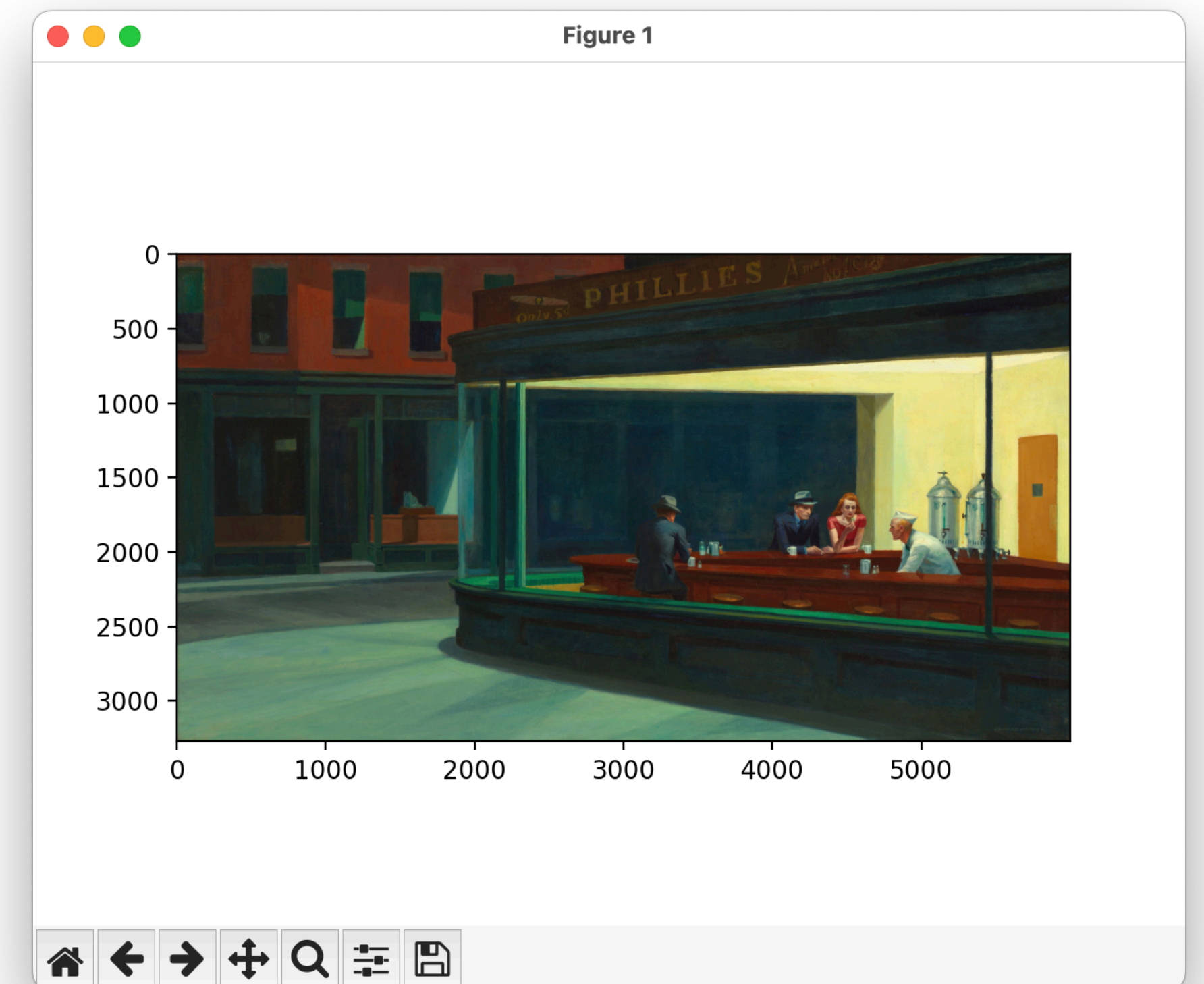
코드 작성하고 실행

- main.py

```
import matplotlib.pyplot as plt
import matplotlib.image as img
import os

base_path = os.path.abspath('.')
path = os.path.join(base_path, 'images', 'Nighthawks.jpeg')
image = img.imread(path)
plt.imshow(image)
plt.show()
```

macOS나 linux 는 경로 구분자로 / 윈도우즈는 \\ 를 사용하므로 경로는
os.path.join을 사용해 합성한다.



<https://artvee.com/dl/nighthawks#00>

앱 용으로 코드 변경

- main.py

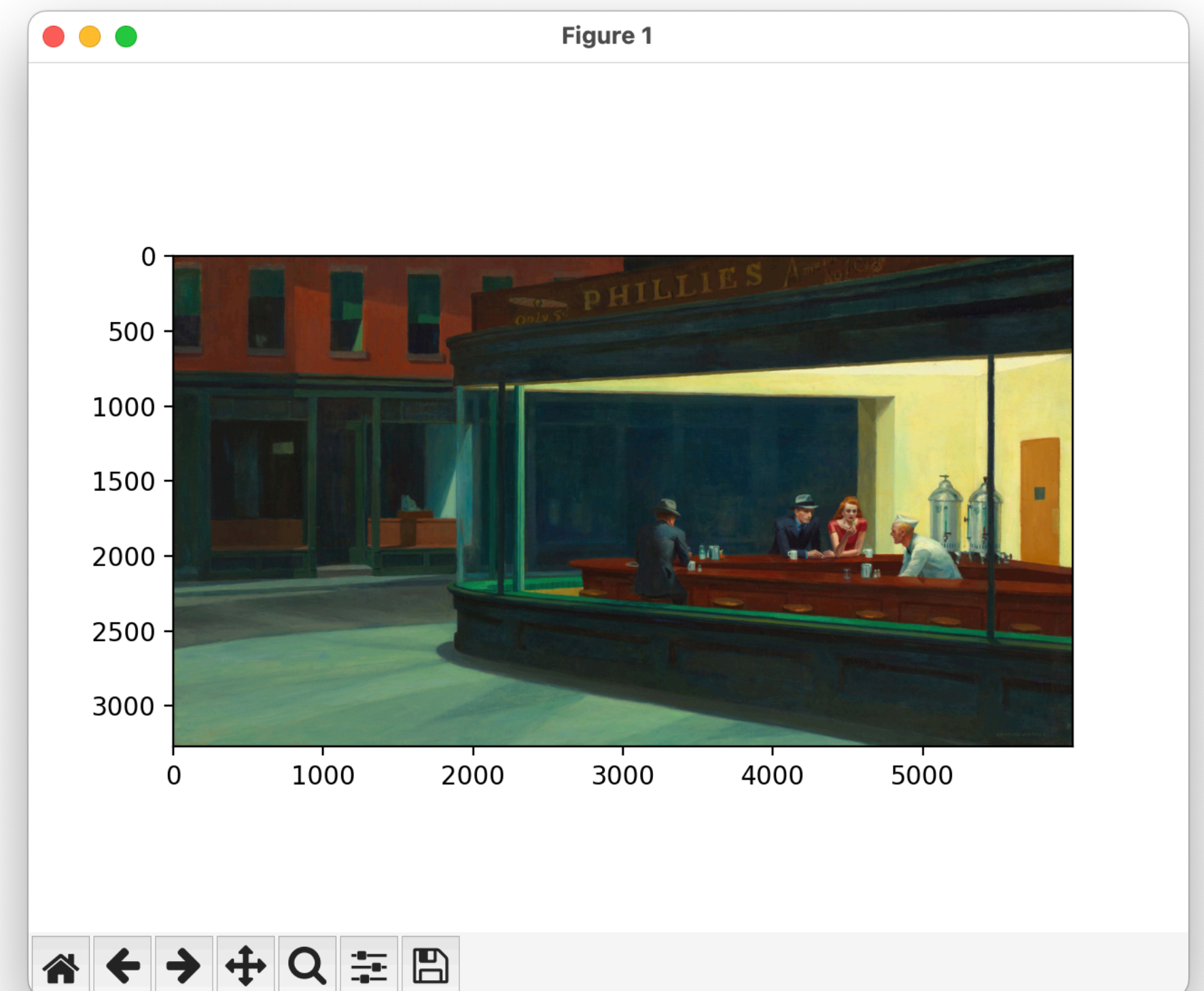
```
import matplotlib.pyplot as plt
import matplotlib.image as img
import sys
import os

def resource_path(relative_path):
    try:
        # 앱이 설치된 폴더의 _internal 폴더 경로
        base_path = sys._MEIPASS
    except Exception:
        base_path = os.path.abspath(".")
    return os.path.join(base_path, relative_path)

path = resource_path(os.path.join("images", "Nighthawks.jpeg"))
image = img.imread(path)
plt.imshow(image)
plt.show()
```

pyinstaller로 생성된 앱은 _MEIPASS 가 있고

소스코드 상태에서는 해당 폴더가 없다.



<https://artvee.com/dl/nighthawks#00>

단독 실행할 앱으로 빌드하기

- 터미널에서 다음 명령어 실행
`pyinstaller main.py`
- build와 dist 폴더가 추가되고 dist 폴더에 앱 파일이 생성된다.
 - 실행 파일과 _internal 파일이 생성된다.
 - 실행 파일과 _internal 폴더는 같은 폴더에 있어야 한다.
- onefile 옵션으로 하나의 파일로 빌드할 수 있지만 이 경우 파일 내부에 함께 저장된 라이브러리를 준비하는 과정에서 시간이 오래 걸려 초기 화면 출력이 매우 느릴 수 있다.

실행이 안되는 이유는?

- 코드가 images 폴더 및 그 내부 파일을 참조한다.
- 앱을 빌드 할 때 images 폴더도 함께 빌드해야 한다.

```
pyinstaller main.py --add-data images/:images/
```

- 윈도우즈는 ; 맥이나 리눅스는 :
- 현재 폴더에 있는 images 폴더를 빌드 된 앱에 images 라는 이름으로 추가해라.

```
pyinstaller --noconfirm main.py --add-data images/:images/
```

- dist 파일에 기존 빌드 결과가 있을때 일일이 묻지말고 빌드해라 라는 뜻.

spec

- 빌드할 때마다 많은 옵션을 적기는 귀찮으므로 pyinstaller용 빌드 스크립트를 만든다.
- 터미널에서 다음 명령어 입력
`pyi-makespec main.py`
- 프로젝트 폴더에 main.spec 파일이 추가된다.

spec

- 파일을 다음과 같이 수정

```
# -*- mode: python ; coding: utf-8 -*-

a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],
    datas=[
        ('images/', 'images/')
    ],
    hiddenimports=[],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
    optimize=0,
)

# 다른 부분은 수정 없음
```

- 이제 다음 명령어로 빌드 가능

pyinstaller --noconfirm main.spec

React + Pywebview

Pywebview

<https://pywebview.flowrl.com/>

- HTML 을 이용해 GUI를 제공하는 파이썬 라이브러리
 - html/css/frontend js - react에서 빌드한 파일 포함
 - 실행중인 서버

- pywebview 설치

```
pip install pywebview
```

Pywebview

<https://pywebview.flowrl.com/>

- 서비스 중인 사이트 사용하기
- app.py

```
import webview
```

```
webview.create_window('Hello world', 'https://pywebview.flowrl.com/')  
webview.start()
```



Pywebview

<https://pywebview.flowrl.com/>

- html 파일 사용하기 - index.html 작성 후 app.py 수정

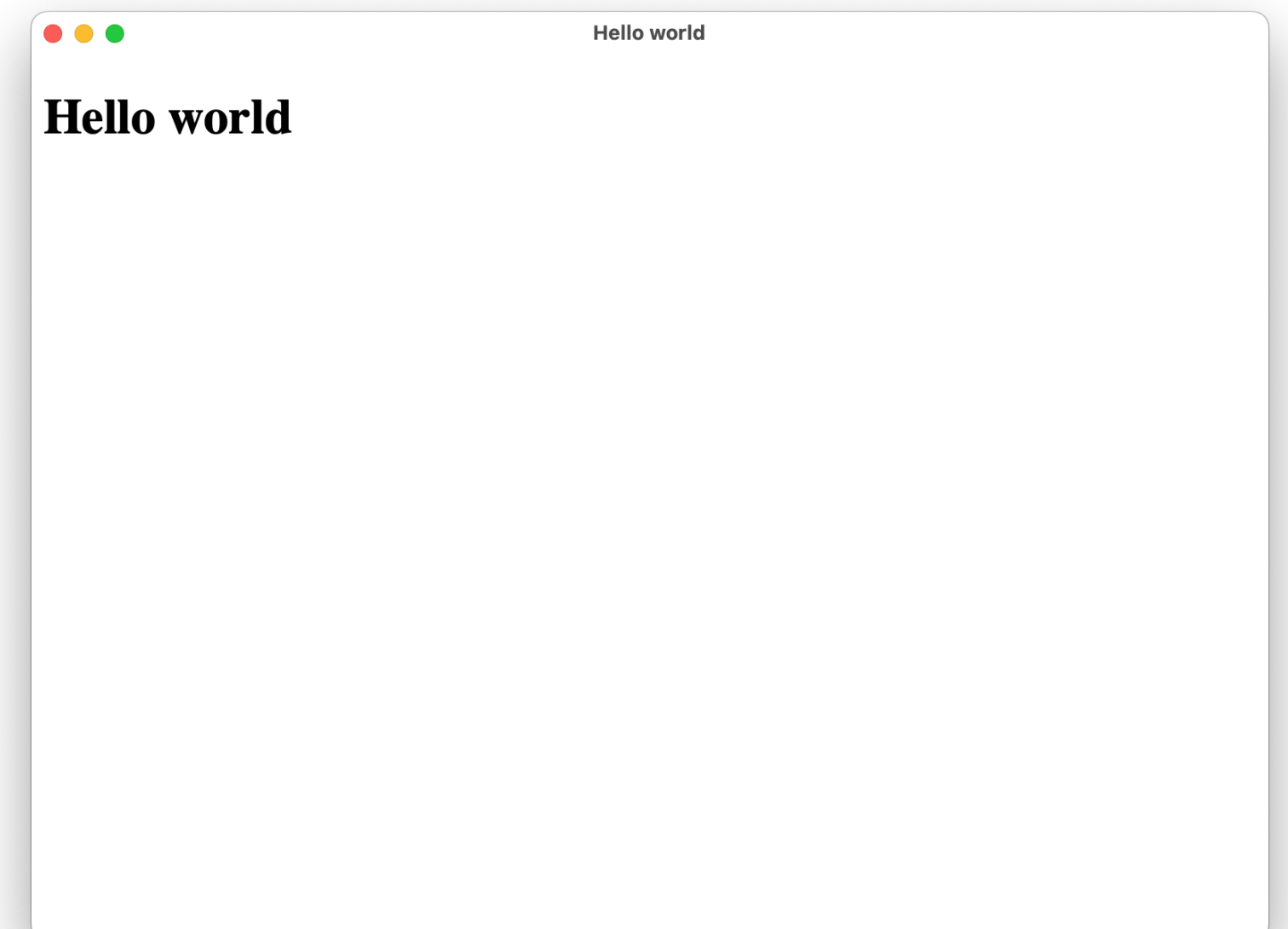
```
<!DOCTYPE html>
<html>
  <head></head>
  <body>
    <h1>Hello world</h1>
  </body>
</html>
```

index.html

```
import webview

webview.create_window('Hello world', 'index.html')
webview.start()
```

app.py



React 와 결합하기

- 프로젝트 루트에서 터미널 명령어로 react 프로젝트 만들기
 - `npm create vite@latest frontend -- --template react`

```
● (.venv) → PITest npm create vite@latest frontend -- --template react

> npx
> "create-vite" frontend --template react

|
|
| Use rolldown-vite (Experimental)?:
| No
|
| Install with npm and start now?
| No
|
| Scaffolding project in /Users/suyeoncho/Codes/Python/PITest/frontend...
|
| Done. Now run:
|
| cd frontend
| npm install
| npm run dev
```


React 와 결합하기

```
cd frontend
```

```
npm i
```

- frontend/src/App.jsx 수정 후 터미널에서 `npm run dev`

```
import './App.css'

function App() {
  return (
    <>
      <h1>Hello World</h1>
    </>
  )
}

export default App
```

UI 관련 작업을 할 때는 frontend 폴더 터미널에서 명령어를 입력해야 한다.

React 와 결합하기

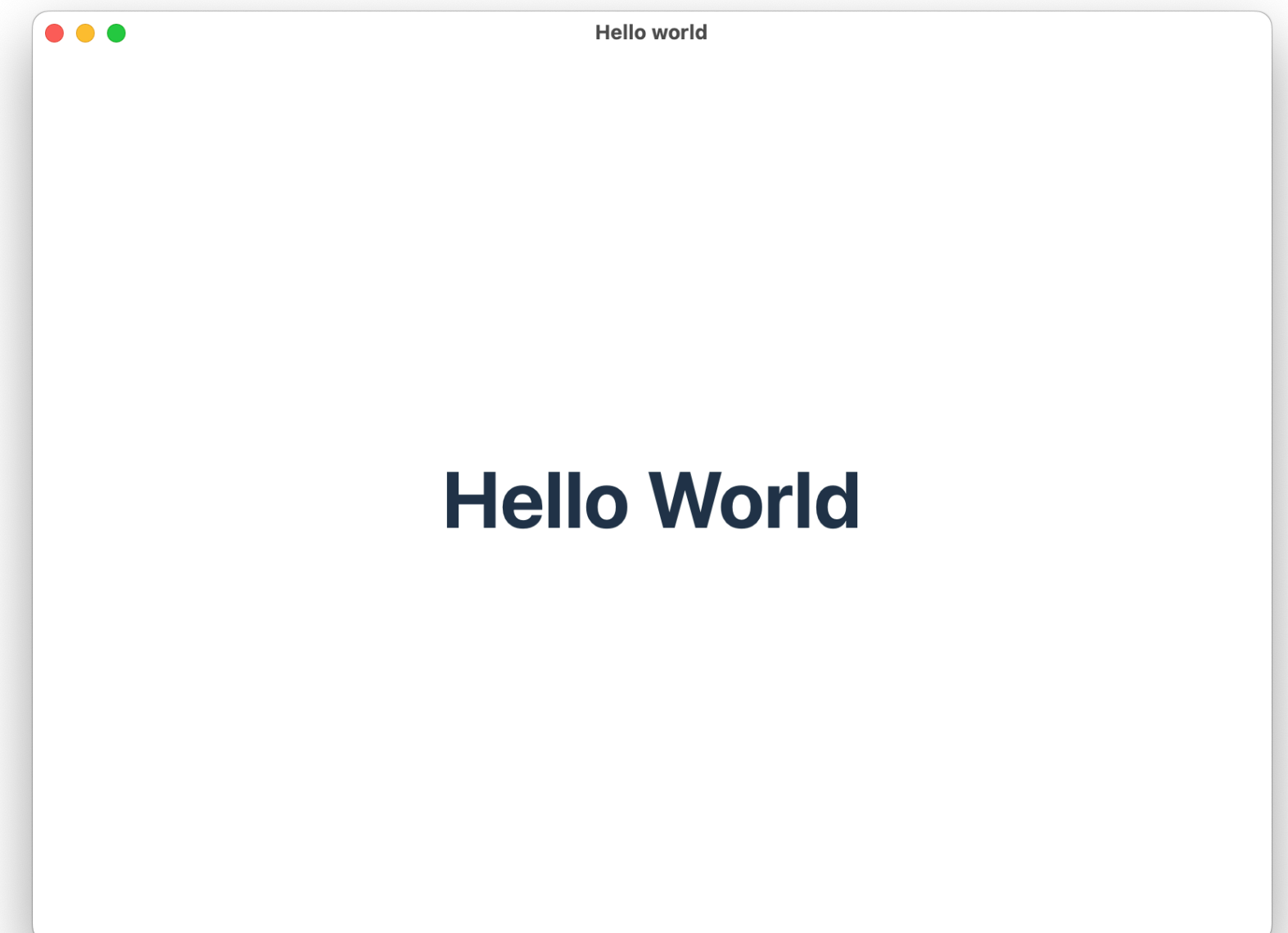
- app.py 수정

```
import webview

webview.create_window('Hello world', 'http://localhost:5173')
webview.start()
```

- 현재 터미널은 리액트 앱이 실행중이므로 새 터미널을 열고

```
python app.py
```



API

- 파이썬의 기능을 리액트에서 사용할 수 있도록 api 를 작성한다.

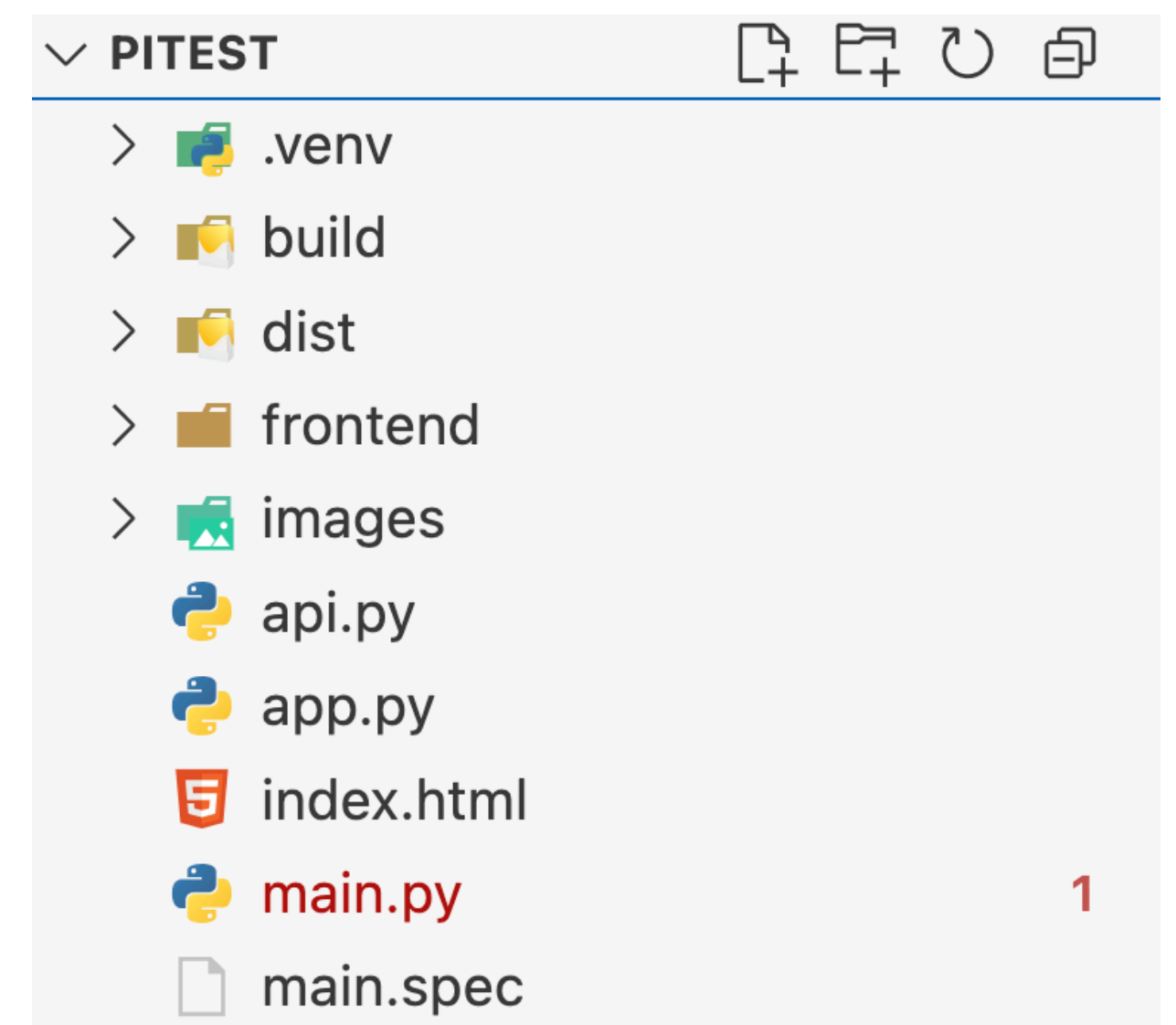
- api.py 파일 작성

```
class Api:
    def get_message(self):
        return 'Hello world!!!!!!'
```

- app.py 수정

```
import webview
from api import Api

webview.create_window(
    'Hello world',
    'http://localhost:5173',
    js_api=Api()
)
webview.start()
```



API

- frontend/src/App.jsx 수정

```
import './App.css'
import { useEffect, useState } from 'react'
function App() {
  const [message, setMessage] = useState('')
  useEffect(()=>{
    const getMessage = async()=>{
      const response = await window.pywebview.api.get_message()
      setMessage(response)
    }
    getMessage()
  }, [])
  return (
    <>
      <h1>{message}</h1>
    </>
  )
}

export default App
```

window.pywebview.api 를 통해 파이썬 함수를 사용할 수 있다.

API

- customHook 추가
 - 앱을 실행하는 PC 가 성능이 좋지 않을 경우 window에 pywebview 가 추가되기도 전에 useEffect가 동작해 버릴 수도 있다. 즉, 파이썬 초기화 전에 UI가 먼저 초기화 해버리는 경우이다.
 - 이를 대비해 pywebview가 제공하는 이벤트를 이용하는 커스텀 훅을 추가한다.
- frontend/src/hooks 폴더 추가
- usePywebviewReady.js 파일 추가

API

- frontend/src/hooks/usePywebviewReady.js

```
import { useState, useEffect } from 'react';

export function usePywebviewReady() {
  const [isReady, setIsReady] = useState(!!window.pywebview);

  useEffect(() => {
    if (isReady) return;

    const handleReady = () => {
      setIsReady(true);
    };
    window.addEventListener('pywebviewready', handleReady);

    return () => {
      window.removeEventListener('pywebviewready', handleReady);
    };
  }, [isReady]);

  return isReady;
}
```


API

- frontend/src/App.jsx 수정

```
import './App.css'
import { useEffect, useState } from 'react'
import { usePywebviewReady } from './hooks/usePywebviewReady'

function App() {
  const isReady = usePywebviewReady();
  const [message, setMessage] = useState('')
  useEffect(() => {
    const getMessage = async () => {
      const response = await window.pywebview.api.get_message()
      setMessage(response)
    }
    if (isReady)
      getMessage()
  }, [isReady])
  return (
    <>
      <h1>{message}</h1>
    </>
  )
}

export default App
```

api 가 준비되면 함수를 호출한다.

UI 빌드 하기

- 실행 중인 리액트 앱을 참조하는 것은 디버깅용.
- exe 로 빌드하기 위해서는 ui 코드를 빌드해 사용해야 한다.
 - frontend 폴더에서 다음 명령어 입력
`npm run build`
- frontend 에 dist 폴더가 추가되고 index.html과 css, js 가 생성된다.
- -> pyinstaller 로 빌드할 때 이 폴더도 추가해야 한다.
- -> app.py도 수정해야 한다.

UI 빌드 하기

- app.py

```
import webview
from api import Api
import os
import sys

def resource_path(relative_path):
    try:
        # 앱이 설치된 폴더의 _internal 폴더 경로
        base_path = sys._MEIPASS
    except Exception:
        base_path = os.path.abspath(".")
    return os.path.join(base_path, relative_path)

dist = os.path.join('frontend', 'dist', 'index.html')
frontend = resource_path(dist)
webview.create_window(
    'Hello world',
    frontend,
    js_api=Api()
)
webview.start()
```

앱 빌드하기

- app.spec

```
# -*- mode: python ; coding: utf-8 -*-
import os

FRONTEND_DIR = os.path.join('frontend', 'dist')

a = Analysis(
    ['app.py'],
    pathex=[],
    binaries=[],
    datas=[
        (FRONTEND_DIR, FRONTEND_DIR)
    ],
    hiddenimports=[],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
    optimize=0,
)
pyz = PYZ(a.pure)

exe = EXE(
    pyz,
    a.scripts,
    [],
    exclude_binaries=True,
    name='app',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    console=True,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)
coll = COLLECT(
    exe,
    a.binaries,
    a.datas,
    strip=False,
    upx=True,
    upx_exclude=[],
    name='app',
)
```

요약

- 개발 중일 때
 - 터미널 두 개를 이용해 react 를 dev 모드로 실행하고
 - pywebivew 는 localhost를 사용
- 배포할 때
 - 리액트 앱일 빌드
 - pywebview 는 빌드 된 앱의 index.html 을 사용

장점과 단점

- 장점

- GUI 에 디자인 적용이 비교적 쉽다
- 리액트용 각종 라이브러리의 기능을 모두 사용하면서도 파이썬 백엔드의 기능을 활용할 수 있다.

- 단점

- 리액트와 파이썬 모두를 알아야 한다.
- 프로젝트 구성이 복잡하다.

Tkinter

Tkinter

<https://docs.python.org/ko/3/library/tkinter.html>

- 장점

- 파이썬에 내장된 GUI 라이브러리 - 즉시 사용할 수 있다.
- 실행 속도가 비교적 빠르다.

- 단점

- 복잡한 화면을 만들기 어렵다.

- 참조

- <https://tkdocs.com/tutorial/firstexample.html>
- <https://blog.naver.com/sisosw/221408280038>

Tkinter

- pyapp.py

```
import tkinter as tk
from api import Api

api = Api()

window = tk.Tk()
window.title('Hello')
window.geometry("600x400") #윈도우 사이즈

message = tk.StringVar(value=api.get_message())

label = tk.Label(
    window,
    textvariable = message,
    font = ("Arial", 40)
)

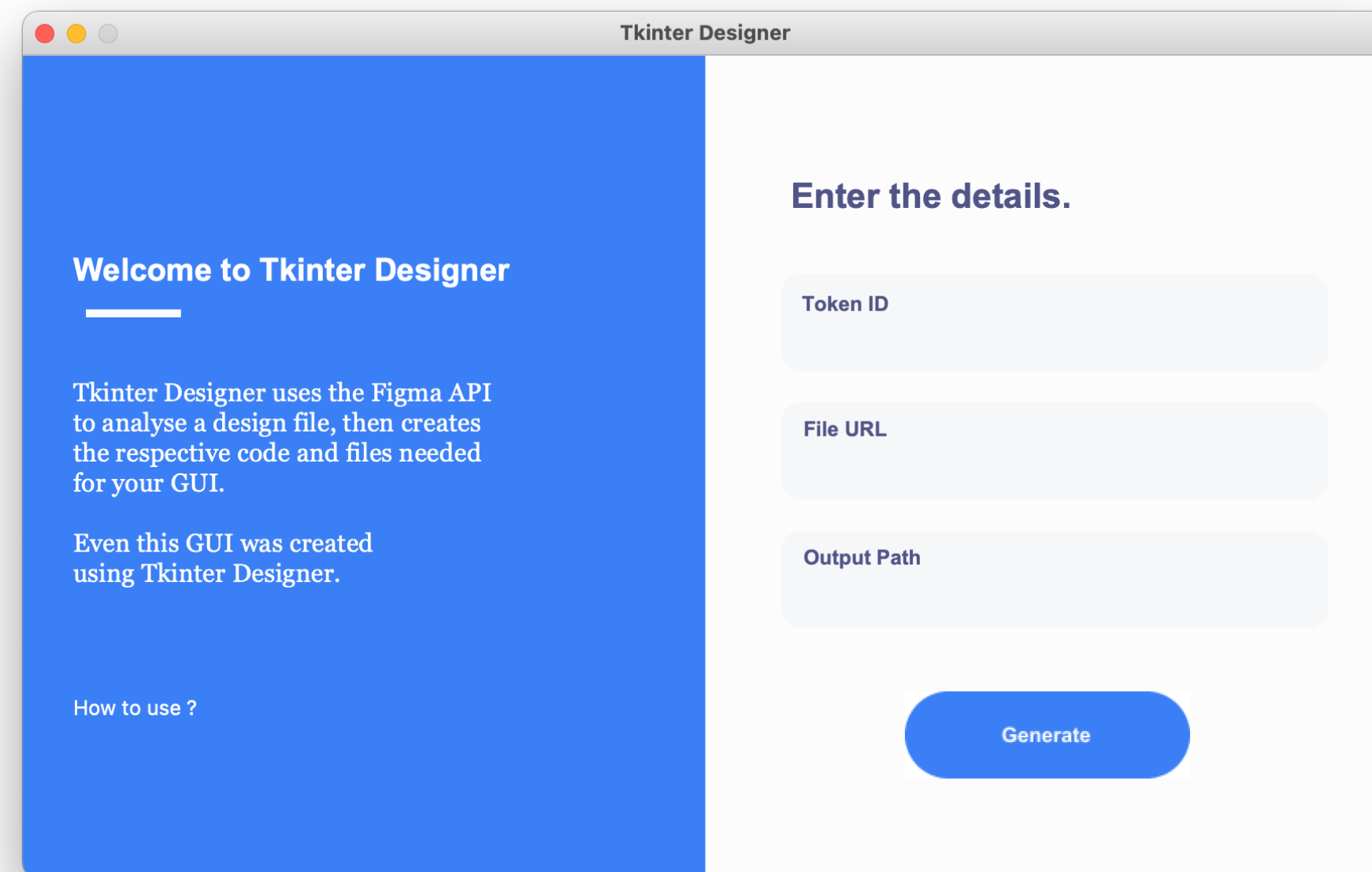
label.pack(pady=10, expand=True)

window.mainloop()
```

Tkinter Designer

<https://github.com/ParthJadhav/Tkinter-Designer>

- 프로젝트에 tkinter-designer를 설치한다.
 - pip install tkdesigner
 - <https://github.com/ParthJadhav/Tkinter-Designer> 의 코드를 다운받아 실행



Tkinter Designer

<https://github.com/ParthJadhav/Tkinter-Designer>

- 피그마에서 GUI를 디자인한다.
 - 단, 레이어 이름을 지정된 이름으로 지어줘야 한다.
 - <https://github.com/ParthJadhav/Tkinter-Designer/blob/master/docs/instructions.kr-KR.md>
- 앞서 실행된 창에 토큰, url, 출력 위치 등을 지정한다.
 - <https://m.blog.naver.com/yug311861/222915865128>